

**SIMATS ENGINEERING**  
**Department of Computer Science & Engineering**  
**ASSESSMENT TOOL 2- INDUSTRY PROBLEM SOLVING TASK**

|                |                                                                                                                                                                                                                                                    |               |                                                       |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------------------------------------------------|
| Course Code:   | CSA1223                                                                                                                                                                                                                                            | Course Title: | Computer Architecture                                 |
| CO Assessed:   | CO4                                                                                                                                                                                                                                                | Assessment:   | ASSESSMENT TOOL2-<br>INDUSTRY PROBLEM<br>SOLVING TASK |
| Weightage:     | 30%                                                                                                                                                                                                                                                | Total Marks:  | 100                                                   |
| CO4 Statement: | Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics. |               |                                                       |
| Student Name:  | M. Alivetha                                                                                                                                                                                                                                        | Reg.No.:      | 192471040                                             |
| Department:    | CS & BS                                                                                                                                                                                                                                            | Date:         |                                                       |

**ANNEXURE A**  
**INDUSTRY PROBLEM SOLVING TASK**

75% 

- Smartphone Performance Optimization**  
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
- Cloud Server Memory Bottleneck**  
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
- Autonomous Vehicle Data Processing**  
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
- AI Inference Edge Device**  
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
- High-Performance Gaming Console**  
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

### Code of Conduct:

I M. Nivedha (1924702) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

### MARKS ALLOCATION

|    | Understanding of Industry Problem (20%) | Application of Engineering Concepts (25%) | Solution Design (25%) | Use of Tools (15%) | Analysis (10%) |      |
|----|-----------------------------------------|-------------------------------------------|-----------------------|--------------------|----------------|------|
| Q1 | 1                                       | 1.25                                      | 0.5                   | 0.5                | 0.75           | 4    |
| Q2 | 1                                       | 1.5                                       | 0.5                   | 0.5                | 0.75           | 3.25 |
| Q3 | 0.5                                     | 1                                         | 1                     | 0.5                | 0.25           | 3.25 |
| Q4 | 0.5                                     | 1                                         | 1                     | 1                  | 0.5            | 4    |
| Q5 | 1                                       | 1.25                                      | 0.5                   | 1                  | 0.5            | 4.25 |

### RUBRICS FOR CALCULATION:

| Criteria                            | Weightage | Level 4 - Exemplary                                                          | Level 3 - Proficient                                 | Level 2 - Developing                           | Level 1 - Beginning                               |
|-------------------------------------|-----------|------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Understanding of Industry Problem   | 20%       | Comprehensive understanding of real-world problem and constraints            | Good understanding with minor gaps in requirements   | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25%       | Applies advanced and relevant concepts effectively to solve the problem      | Applies appropriate concepts with minor errors       | Limited application of concepts                | Incorrect or no application of concepts           |
| Solution Design                     | 25%       | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps        | Basic solution with limited feasibility        | Unclear or impractical solution                   |
| Use of Tools                        | 15%       | Effective use of modern tools, technologies, and real data                   | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools           | Minimal or incorrect use of tools                 |
| Analysis                            | 10%       | Thorough analysis with validated results and performance evaluation          | Good analysis with reasonable validation             | Basic analysis with limited validation         | Poor or no analysis                               |

18.75



**SIMATS ENGINEERING**  
**Department of Computer Science & Engineering**  
**ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK**

---

|                |                                                                                                                                                                                                                                                           |               |                                                       |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------------------------------------------------|
| Course Code:   | CSA1223                                                                                                                                                                                                                                                   | Course Title: | Computer Architecture                                 |
| CO Assessed:   | CO4                                                                                                                                                                                                                                                       | Assessment:   | ASSESSMENT TOOL1-<br>INDUSTRY PROBLEM<br>SOLVING TASK |
| Weightage:     | 30%                                                                                                                                                                                                                                                       | Total Marks:  | 100                                                   |
| CO4 Statement: | <i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i> |               |                                                       |
| Student Name:  |                                                                                                                                                                                                                                                           | Reg.No.:      |                                                       |
| Department:    |                                                                                                                                                                                                                                                           | Date:         |                                                       |

**ANNEXURE A**  
**INDUSTRY PROBLEM SOLVING TASK**

**1. Smartphone Performance Optimization**

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

**Smartphone Performance Optimization**

**1. Understanding of the Industry Problem**

A smartphone manufacturer observes **lag while switching between applications**. Modern smartphones run multiple applications simultaneously, requiring frequent access to instructions, data, and operating-system processes. If the required data is not available in fast cache memory, the processor must access slower DRAM, increasing memory access latency.

The main objective is to design an efficient **hierarchical memory system using L1, L2, L3 cache and DRAM** to reduce latency and improve application-switching performance.

**2. Application of Engineering Concepts**

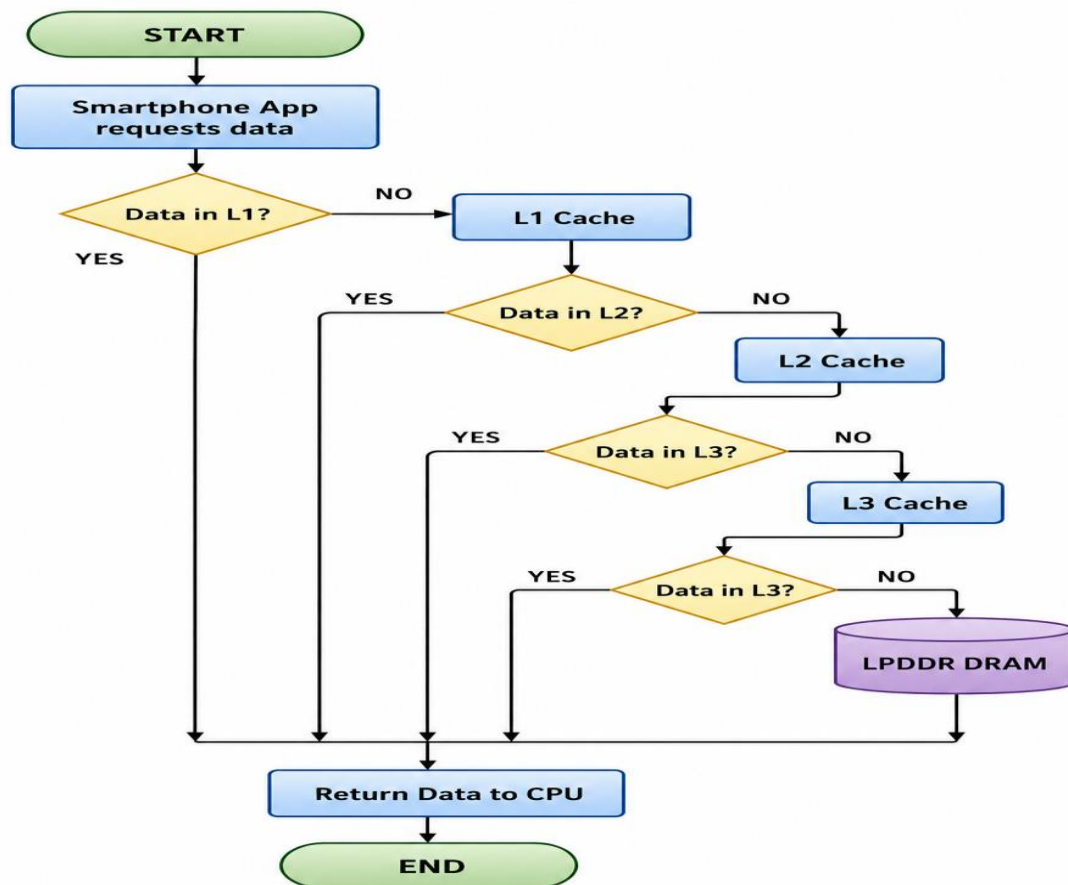
The memory hierarchy can be arranged as:

**CPU → L1 Cache → L2 Cache → L3 Cache → DRAM**

| Memory   | Speed     | Capacity     | Main Purpose                      |
|----------|-----------|--------------|-----------------------------------|
| L1 Cache | Very High | Very Small   | Frequently used instructions/data |
| L2 Cache | High      | Small/Medium | Backup for L1 misses              |
| L3 Cache | Moderate  | Larger       | Shared cache between cores        |
| DRAM     | Lower     | Large        | Main memory for applications      |

- **L1 cache** has the lowest latency and is closest to the CPU core.
- **L2 cache** provides additional storage with slightly higher latency.
- **L3 cache** is larger and is generally shared among processor cores.
- **DRAM** provides large capacity but has higher access latency.

## 2.Proposed Hierarchical Memory Redesign



*Proposed Hierarchical Memory Redesign for Smartphone Performance Optimization*

## Redesign:

1. **Optimize L1 Cache** – Improve L1 efficiency for faster access to frequently used data.
2. **Dedicated L2 Cache** – Provide sufficient L2 cache for each CPU core.
3. **Shared L3 Cache** – Allow cores to share data and reduce DRAM accesses.
4. **High-Bandwidth LPDDR DRAM** – Enable faster data transfer between CPU and main memory.
5. **Cache Frequent Data** – Keep frequently used application data in cache to reduce delays.
6. **Multicore Processing** – Use multiple CPU cores to handle applications simultaneously.
7. **Improved Performance** – Reduce memory latency, improve CPU utilization, and provide faster response.

## 4. Use of Tools and Technologies

For this project, the following tools can be used:

| Tool                    | How it is Used in This Problem                                                                                                |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Python                  | Build a cache-hierarchy simulation for L1, L2, L3 and DRAM and calculate cache hit rate, miss rate, AMAT, and memory latency. |
| Google Colab            | Run the Python simulation and test different cache sizes and configurations.                                                  |
| Android Studio Profiler | Measure CPU usage, memory usage, and application performance while switching between applications.                            |
| Matplotlib              | Generate graphs comparing cache hit rate, memory latency, DRAM accesses, and application-switching time.                      |

## How the Tools Are Applied

### Step 1 – Collect Application Workload

Use **Android Studio Profiler** to observe CPU and memory behavior during application switching.

### Step 2 – Create Memory Simulation

Use **Python** to model:

**CPU → L1 → L2 → L3 → DRAM**

The simulation generates memory requests and determines whether each request results in a **cache hit or cache miss**.

### Step 3 – Run Experiments

Use **Google Colab** to test:

- Existing cache configuration
- Proposed cache configuration
- Different cache sizes
- Different workloads

### Step 4 – Calculate Performance

Python calculates:

Cache Hit Rate  
Cache Miss Rate  
AMAT  
DRAM Accesses  
Memory Latency

Using:

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

### Step 5 – Generate Graphs

Use **Matplotlib** to compare:

#### Existing Design vs Proposed Design

For example:

- Cache hit rate ↑
- Cache misses ↓
- DRAM accesses ↓
- Memory latency ↓
- Application-switching time ↓
- Throughput ↑

## 5. Analysis and Performance Evaluation

The redesign reduces the number of accesses reaching DRAM. A higher cache hit rate allows frequently used data to be obtained from faster memory.

The average memory access time can be represented as:

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Therefore:

**Higher Cache Hit Rate → Fewer DRAM Accesses → Lower Memory Latency → Faster Application Switching**

## 6. Result

The proposed **L1–L2–L3–DRAM hierarchical memory system** improves smartphone responsiveness by keeping frequently accessed data closer to the CPU. L1 and L2 reduce access latency, while L3 reduces expensive DRAM accesses.

**Conclusion:** An optimized cache hierarchy combined with high-bandwidth DRAM can reduce smartphone lag and improve overall application-switching performance.

## 2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

### Cloud Server Memory Bottleneck

#### 1. Understanding of the Industry Problem

A cloud service provider experiences **high latency during database queries**. Database systems frequently access large datasets, indexes, transaction records, and intermediate results. If the required data is not available in cache or physical memory, the system may access virtual memory through paging, which significantly increases latency.

The objective is to design an efficient memory system that reduces **cache misses, page faults, and database query latency**.

## 2. Cache Hierarchy vs Virtual Memory Paging

| Feature          | Cache Hierarchy                  | Virtual Memory Paging   |
|------------------|----------------------------------|-------------------------|
| Main Purpose     | Reduce CPU memory access latency | Extend available memory |
| Managed By       | Hardware                         | OS + Hardware           |
| Speed            | Very High                        | Much Slower             |
| Storage          | L1, L2, L3 Cache                 | DRAM + SSD              |
| Main Problem     | Cache Miss                       | Page Fault              |
| Database Benefit | Faster repeated access           | Allows larger datasets  |

**Cache hierarchy** stores frequently accessed data close to the processor. **Virtual memory paging** allows programs to use more memory than physically available in DRAM, but page faults can cause access to much slower storage.

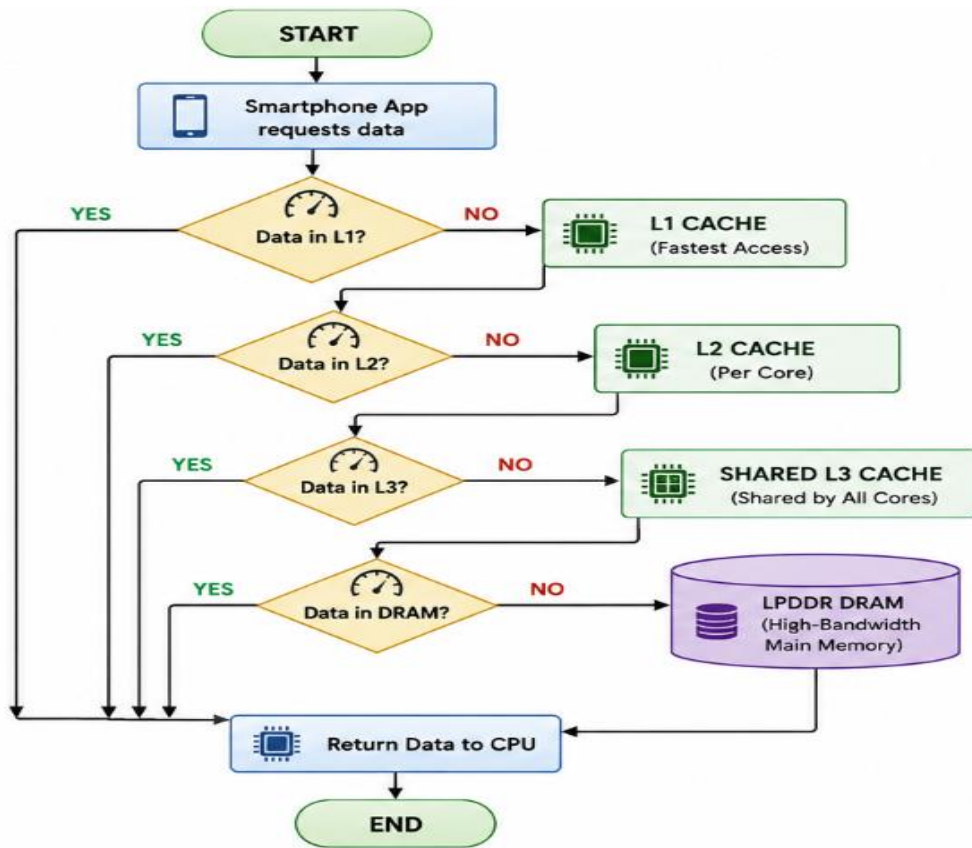
## 3. Proposed Memory Architecture

The proposed cloud-server memory hierarchy is:

**CPU Cores → L1 Cache → L2 Cache → Shared L3 Cache → Large DRAM → NVMe SSD**

**Process flow:**





**Figure 2: Proposed Memory Architecture for Cloud Database Performance**

#### 4. Solution Design

The following improvements are recommended:

1. Increase effective **L3 cache capacity**.
2. Provide sufficient **DRAM** for active database workloads.
3. Reduce unnecessary page faults.
4. Optimize database indexes and queries.
5. Use high-speed **NVMe SSD** for persistent storage.
6. Monitor memory utilization continuously.
7. Keep frequently accessed database records in faster memory.

#### 5. Use of Tools and Technologies

For this project, the following tools can be used:

| Tool         | How it is Used in This Problem                                                                                                                                              |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Python       | Build a memory-hierarchy simulation for L1, L2, L3 cache, DRAM, and virtual memory paging and calculate cache misses, page faults, memory latency, and query response time. |
| Google Colab | Run the Python simulation and test different cache, DRAM, and paging configurations.                                                                                        |
| Linux Perf   | Monitor cache misses, CPU performance, and memory-related events during database workloads.                                                                                 |
| Matplotlib   | Generate graphs comparing cache miss rate, page-fault rate, memory latency, and database query latency.                                                                     |

## How the Tools Are Applied

### Step 1 – Generate Database Workload

Use **Python** to generate database-like memory access patterns such as repeated access to indexes, records, and transaction data.

### Step 2 – Create Memory Simulation

Use **Python** to model:

**CPU → L1 → L2 → L3 → DRAM → NVMe SSD**

The simulation identifies **cache hits, cache misses, and page faults**.

### Step 3 – Run Experiments

Use **Google Colab** to test:

- Existing memory configuration
- Proposed memory configuration
- Different cache sizes
- Different DRAM capacities
- Different paging conditions

## Step 4 – Monitor Performance

Use **Linux Perf** to observe:

- Cache misses
- CPU utilization
- Memory accesses
- Performance-counter data

## Step 5 – Calculate Performance

Python calculates:

**Cache Miss Rate**

**Page-Fault Rate**

**Memory Latency**

**Database Query Latency**

**Memory Utilization**

## Step 6 – Generate Graphs

Use **Matplotlib** to compare:

### Existing Design vs Proposed Design

For example:

- Cache misses ↓
- Page faults ↓
- DRAM accesses ↓
- Query latency ↓
- Memory utilization ↑
- Database throughput ↑

## 6. Analysis

Database workloads frequently access indexes and repeatedly used records. These benefit from cache memory because the data is located close to the CPU.

If the database working set is larger than available DRAM, the operating system may perform paging, causing slower storage accesses.

The performance relationship is:

**Higher Cache Hit Rate → Fewer DRAM Accesses → Fewer Page Faults → Lower Query Latency**

Therefore, increasing **effective cache utilization and sufficient DRAM capacity** is more beneficial than depending heavily on virtual-memory paging.

## 7. Result

The proposed **L1–L2–L3 cache + large DRAM + NVMe SSD** architecture reduces cache misses and minimizes virtual-memory paging. This improves database query response time and overall cloud-server performance.

**Conclusion:** A balanced memory hierarchy with efficient cache utilization, sufficient DRAM, and fast NVMe storage provides an effective solution for reducing database latency in cloud servers.

## 3. Autonomous Vehicle Data Processing

**An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.**

### Autonomous Vehicle Data Processing

#### 1. Understanding of the Industry Problem

An autonomous vehicle continuously receives large amounts of data from **cameras, LiDAR, radar, GPS, and other sensors**. This information must be processed in real time for object detection, vehicle localization, path planning, and safety decisions.

The major requirements are:

- Very low processing latency
- High memory bandwidth
- Fast sensor-data access
- Reliable real-time operation
- Efficient multicore processing

Therefore, an appropriate memory hierarchy using **SRAM, DRAM, and MRAM** is required.

## 2. Comparison of SRAM, DRAM and MRAM

| Feature    | SRAM      | DRAM        | MRAM                     |
|------------|-----------|-------------|--------------------------|
| Speed      | Very High | High        | High                     |
| Capacity   | Low       | High        | Medium                   |
| Volatility | Volatile  | Volatile    | Non-volatile             |
| Cost/bit   | High      | Lower       | Higher                   |
| Main Use   | CPU Cache | Main Memory | Fast Non-volatile Memory |

### SRAM

SRAM provides **very low latency** and is suitable for CPU and AI-accelerator caches. However, it has limited capacity and higher cost per bit.

### DRAM

DRAM provides **large capacity and high bandwidth**, making it suitable for storing large sensor datasets, AI models, and intermediate processing data.

### MRAM

MRAM is **non-volatile** and can retain data without continuous power. It can be used for selected persistent data and applications requiring fast non-volatile memory.

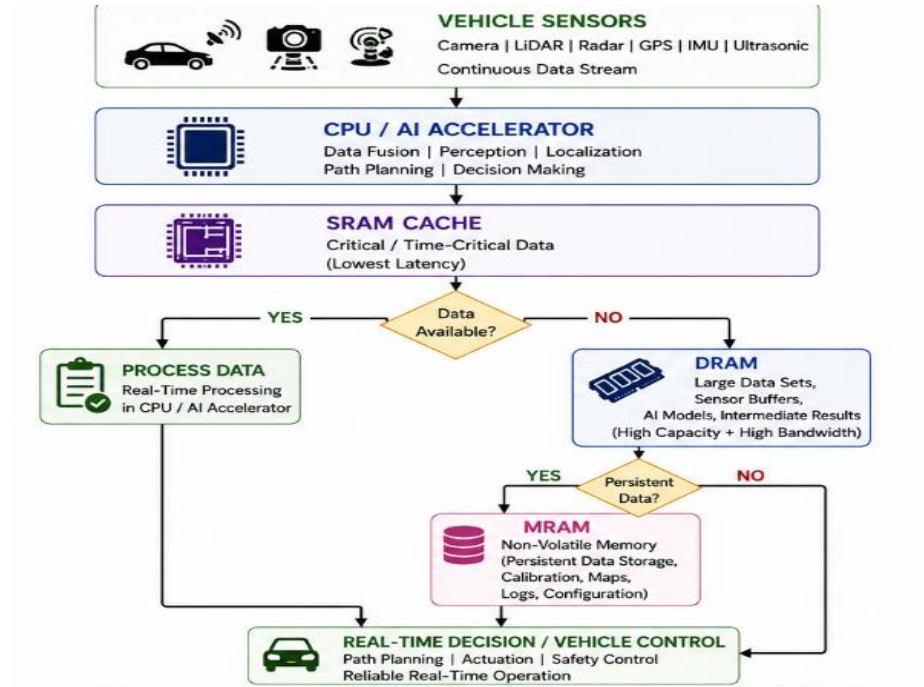
## 3. Proposed Memory Hierarchy

The proposed autonomous vehicle memory architecture is:

**Sensors → CPU/AI Accelerator → SRAM Cache → DRAM → MRA**



## Process Flow



**Figure 3: Proposed Memory Architecture for Autonomous Vehicle Data Processing**

## 4. Solution Design

The proposed system uses:

1. **SRAM** for time-critical data and processor cache.
2. **DRAM** for large sensor buffers and AI workloads.
3. **MRAM** for selected non-volatile data.
4. Multiple memory channels to improve bandwidth.
5. Multicore processors for parallel sensor-data processing.
6. Efficient data movement between sensors, memory, and AI accelerators.

## 5. Use of Tools and Technologies

For this project, the following tools can be used:

| <b>Tool</b>     | <b>How it is Used in This Problem</b>                                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Python          | Build a memory-hierarchy simulation for SRAM, DRAM, and MRAM and calculate memory latency, bandwidth, throughput, and sensor-processing time. |
| Google Colab    | Run the Python simulation and test different memory configurations and sensor workloads.                                                      |
| CARLA Simulator | Generate autonomous-vehicle sensor workloads such as camera, LiDAR, radar, and GPS data for testing.                                          |
| Matplotlib      | Generate graphs comparing memory latency, bandwidth, throughput, and sensor-processing time.                                                  |

### How the Tools Are Applied

#### Step 1 – Generate Sensor Workload

Use **CARLA Simulator** to generate:

**Camera + LiDAR + Radar + GPS Data**

#### Step 2 – Create Memory Simulation

Use **Python** to model:

**Sensors → CPU/AI Accelerator → SRAM → DRAM → MRAM**

The simulation determines how sensor data is stored and accessed at each memory level.

#### Step 3 – Run Experiments

Use **Google Colab** to test:

- Existing memory configuration
- Proposed memory configuration
- Different sensor-data workloads
- Different SRAM/DRAM/MRAM capacities

## Step 4 – Calculate Performance

Python calculates:

Memory Latency

Memory Bandwidth

Data-Processing Throughput

Sensor-Processing Time

CPU Utilization

## Step 5 – Generate Graphs

Use **Matplotlib** to compare:

### Existing Design vs Proposed Design

For example:

- Memory latency ↓
- Sensor-processing time ↓
- Bandwidth ↑
- Throughput ↑
- CPU utilization improved
- Real-time processing performance ↑

## 6. Analysis

Autonomous vehicles require **fast and predictable data processing**. SRAM provides the lowest latency for critical operations, while DRAM provides the capacity needed for large sensor datasets.

MRAM adds non-volatile storage capability for selected data that must be retained without continuous power.

The proposed relationship is:

- **SRAM → Lowest Latency**
- **DRAM → High Capacity + High Bandwidth**
- **MRAM → Non-Volatile Data Storage**

Thus, combining these technologies provides a balanced solution for real-time autonomous vehicle processing.

7. Result

The proposed **SRAM–DRAM–MRAM memory architecture** places critical data close to the processor while providing sufficient capacity for large sensor workloads. This reduces memory access latency and improves real-time processing performance.

**Conclusion:** A hybrid memory architecture using SRAM for fast cache access, DRAM for high-capacity working memory, and MRAM for selected non-volatile data provides an effective solution for autonomous vehicle data processing.

4 .AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

AI Inference Edge Device

1. Understanding of the Industry Problem

An edge AI device needs to **load machine-learning models quickly** while maintaining low power consumption. AI models can be large, and repeatedly transferring them from slower storage to working memory increases model-loading time and energy consumption.

The objective is to design an efficient memory hierarchy using **NAND Flash, DRAM, and RRAM** to achieve fast AI inference with lower power consumption.

2. Comparison of NAND Flash, DRAM and RRAM

| Feature    | NAND Flash   | DRAM     | RRAM         |
|------------|--------------|----------|--------------|
| Speed      | Moderate     | High     | High         |
| Capacity   | Very High    | High     | Medium       |
| Volatility | Non-volatile | Volatile | Non-volatile |

| Feature  | NAND Flash        | DRAM                | RRAM                     |
|----------|-------------------|---------------------|--------------------------|
| Power    | Low standby       | Higher active power | Low potential            |
| Main Use | Permanent Storage | Working Memory      | Fast Non-volatile Memory |

### NAND Flash

NAND Flash provides **high-capacity, non-volatile storage**. It is suitable for permanently storing AI models, applications, and system software.

### DRAM

DRAM provides **fast read/write access** and is used as the working memory for active AI models during inference.

### RRAM

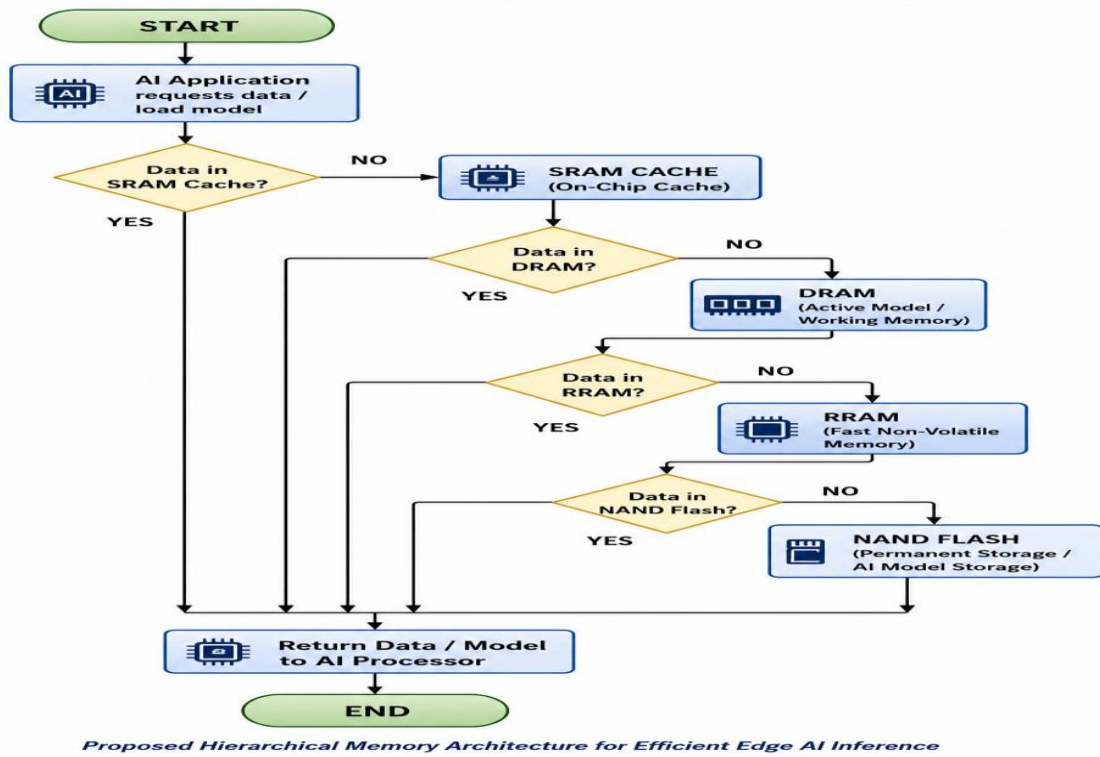
RRAM is a **non-volatile memory technology** that offers potential for fast and energy-efficient access. It can be used for frequently accessed AI models or selected model parameters.

## 3. Proposed Hierarchical Memory Redesign

The proposed architecture is:

**AI Model Storage** → **RRAM** → **DRAM** → **AI Accelerator** with **on-chip SRAM cache** close to the processor.

### Process Flow



**Figure 4: Proposed Hierarchical Memory Architecture for Edge AI Inference**

#### 4. Solution Design

The proposed solution includes:

1. Store permanent AI models in **NAND Flash**.
2. Keep frequently used models in **RRAM** where suitable.
3. Load the active model into **DRAM** for execution.
4. Use **SRAM cache** for frequently accessed model parameters.
5. Reduce unnecessary data transfers between storage and memory.
6. Use model compression and quantization to reduce memory requirements.
7. Optimize memory access to reduce energy consumption.

#### 5. Use of Tools and Technologies

For this project, the following tools can be used:



## Tool / Technology How it is Used in This Problem

|                     |                                                                                                                                                                               |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Python</b>       | Simulate the <b>NAND Flash → RRAM → DRAM → SRAM</b> memory hierarchy and calculate model-loading time, inference latency, memory access time, and energy-related performance. |
| <b>Google Colab</b> | Run the Python simulation and test different memory configurations and AI workloads.                                                                                          |
| <b>PyTorch</b>      | Load and execute AI models and measure model-loading and inference performance.                                                                                               |
| <b>Matplotlib</b>   | Generate graphs comparing model-loading time, inference latency, memory bandwidth, and energy consumption.                                                                    |

## How the Tools Are Applied

### Step 1 – Select AI Workload

Use **PyTorch** to load an AI model such as an image-classification model.

### Step 2 – Create Memory Simulation

Use **Python** to model:

**NAND Flash → RRAM → DRAM → SRAM → AI Accelerator**

The simulation represents movement of the AI model through different memory levels.

### Step 3 – Run Experiments

Use **Google Colab** to test:

- Existing memory configuration
- Proposed memory configuration
- Different model sizes
- Different memory configurations

## Step 4 – Calculate Performance

Python calculates:

Model Loading Time

Inference Latency

Memory Bandwidth

Power Consumption

Energy per Inference

## Step 5 – Generate Graphs

Use **Matplotlib** to compare:

### Existing Design vs Proposed Design

For example:

- Model loading time ↓
- Inference latency ↓
- Data transfers ↓
- Energy consumption ↓
- Memory utilization improved

## 6. Analysis

NAND Flash provides high capacity but has higher access latency than DRAM. DRAM provides fast working memory but is volatile. RRAM provides a potential fast, non-volatile layer for frequently accessed AI data.

The optimized data movement is:

**NAND Flash → RRAM → DRAM → SRAM → AI Accelerator**

Frequently accessed information is kept closer to the processor, reducing repeated access to slower storage.

Therefore:

**Better Memory Placement → Less Data Transfer → Lower Latency → Lower Energy Consumption**

7. Result

The proposed **NAND Flash–RRAM–DRAM–SRAM hierarchy** combines high storage capacity, fast working memory, and non-volatile memory. It reduces model-loading time and unnecessary data movement while supporting efficient AI inference.

**Conclusion:** A hierarchical memory architecture using NAND Flash for permanent storage, RRAM for selected frequently accessed data, DRAM for active models, and SRAM for processor-level caching provides an effective solution for low-latency and energy-efficient edge AI inference.

5.High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

High-Performance Gaming Console

1. Understanding of the Industry Problem

A high-performance gaming console experiences frame drops while loading large game scenes. Modern games contain high-resolution textures, 3D models, audio files, physics data, and other large assets. These workloads require high memory bandwidth and fast data transfer. The main objective is to optimize the L3 cache and DRAM throughput to reduce memory stalls and maintain a stable frame rate.

2. L3 Cache vs DRAM

| Feature      | L3 Cache               | DRAM                        |
|--------------|------------------------|-----------------------------|
| Latency      | Lower                  | Higher                      |
| Capacity     | Smaller                | Much Larger                 |
| Cost per Bit | Higher                 | Lower                       |
| Location     | Close to CPU           | Main Memory                 |
| Main Purpose | Frequently reused data | Large game workloads        |
| Bandwidth    | High                   | Very High overall bandwidth |

**L3 cache** stores frequently accessed data close to the CPU and reduces the need to access DRAM. **DRAM** provides much larger capacity and high bandwidth, making it suitable for large game textures, models, scenes, and other assets. Game performance improves when frequently used data can be accessed quickly from cache.

Textures, 3D models, maps, and game assets are mainly stored in DRAM before being processed by the CPU/GPU.

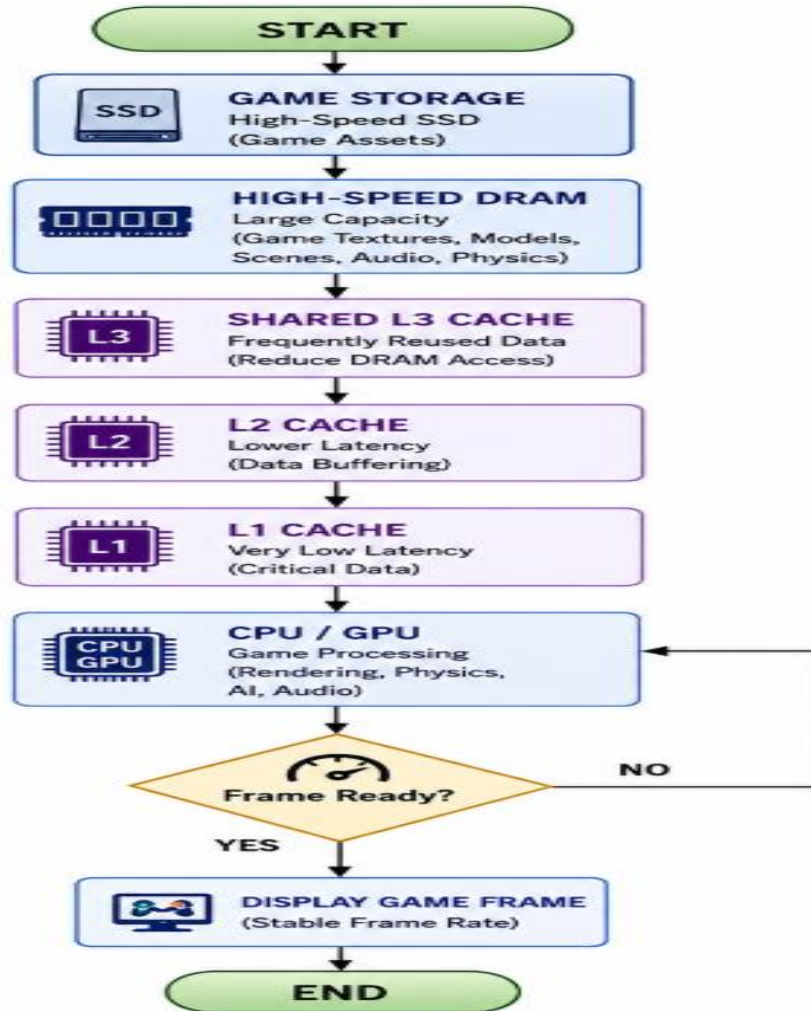
L3 cache helps reduce CPU waiting time and improves overall system performance.

### 3. Proposed Memory Hierarchy

The proposed gaming console architecture is

**Game Storage → DRAM → L3 Cache → L2 Cache → L1 Cache → CPU/GPU**

#### Process Flow



**Figure 5: Proposed Memory Hierarchy for High-Performance Gaming Console**

### 4. Solution Design

The following improvements are recommended:

1. Optimize the **L3 cache** for frequently reused game data.
2. Increase **DRAM bandwidth** using multiple memory channels.
3. Use high-speed memory suitable for GPU workloads.
4. Use high-speed SSD storage for large game assets.
5. Use efficient asset streaming instead of loading everything simultaneously.
6. Apply prefetching to load required data before it is needed.
7. Use texture and asset compression to reduce memory requirements.

## 5. Use of Tools and Technologies

For this project, the following tools can be used:

| Tool                             | How it is Used in This Problem                                                                                                           |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Python                           | Simulate the L1, L2, L3 cache and DRAM hierarchy and analyze cache hit rate, DRAM bandwidth, memory latency, and frame-time performance. |
| Google Colab                     | Run the Python simulation and test different L3 cache sizes and DRAM bandwidth configurations.                                           |
| GPUView / NVIDIA Nsight Graphics | Analyze GPU workload, memory transfers, frame rendering, and GPU performance during large-scene loading.                                 |
| Matplotlib                       | Generate graphs comparing FPS, frame time, L3 cache hit rate, DRAM bandwidth, and scene-loading time.                                    |

### How the Tools Are Applied

#### Step 1 – Generate Gaming Workload

Use a **large-scene gaming workload** containing textures, 3D models, maps, audio, and physics data.

#### Step 2 – Create Memory Simulation

Use **Python** to model:

**Game Data → DRAM → L3 Cache → L2 Cache → L1 Cache → CPU/GPU**

The simulation measures cache hits, cache misses, memory latency, and DRAM data transfers.

#### Step 3 – Run Experiments

Use **Google Colab** to test:

- Existing L3 cache configuration
- Proposed L3 cache configuration
- Different DRAM bandwidths
- Different scene sizes
- Different workload conditions

#### Step 4 – Monitor GPU Performance

Use **NVIDIA Nsight Graphics** or **GPUView** to observe:

- GPU workload
- Memory transfers
- Frame rendering

- Frame-time behavior

## Step 5 – Calculate Performance

Measure:

**FPS**

**FrameTime**

**L3CacheHitRate**

**DRAMBandwidth**

**SceneLoadingTime**

**Memory Latency**

## Step 6 – Generate Graphs

Use **Matplotlib** to compare:

### Existing Design vs Proposed Design

For example:

- L3 hit rate ↑
- DRAM bandwidth utilization ↑
- Memory latency ↓
- Scene loading time ↓
- Frame-time spikes ↓
- FPS stability ↑

## 6. Analysis

Increasing L3 cache can reduce CPU memory latency for frequently reused game data. However, L3 cache has limited capacity and cannot store an entire large game scene.

Therefore, **high-bandwidth DRAM** is essential for transferring large textures, models, and other game assets.

The performance relationship is:

**Higher L3 Hit Rate → Fewer DRAM Accesses → Lower CPU Memory Latency**

and

**Higher DRAM Bandwidth → Faster Asset Transfer → Fewer Memory Stalls → Stable Frame Rate**

Hence, both L3 cache and high-bandwidth DRAM are required for better gaming performance.

## 7. Result

The proposed memory hierarchy combines **optimized L1/L2/L3 caches, high-bandwidth DRAM, and high-speed SSD storage**. L3 cache handles frequently reused data, while DRAM provides the capacity and bandwidth required for large game scenes.

**Conclusion:** A balanced memory architecture using efficient L3 cache, high-bandwidth DRAM, fast SSD storage, asset streaming, and prefetching can reduce frame drops and improve overall gaming performance.

**Code of Conduct:**

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**MARKS ALLOCATION**

|    | Understanding of Industry Problem (20%) | Application of Engineering Concepts (25%) | Solution Design (25%) | Use of Tools (15%) | Analysis (10%) |
|----|-----------------------------------------|-------------------------------------------|-----------------------|--------------------|----------------|
| Q1 |                                         |                                           |                       |                    |                |
| Q2 |                                         |                                           |                       |                    |                |
| Q3 |                                         |                                           |                       |                    |                |
| Q4 |                                         |                                           |                       |                    |                |
| Q5 |                                         |                                           |                       |                    |                |

**RUBRICS FOR CALCULATION:**

| Criteria                            | Weightage | Level 4 - Exemplary                                                          | Level 3 - Proficient                                 | Level 2 - Developing                           | Level 1 - Beginning                               |
|-------------------------------------|-----------|------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Understanding of Industry Problem   | 20%       | Comprehensive understanding of real-world problem and constraints            | Good understanding with minor gaps in requirements   | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25%       | Applies advanced and relevant concepts effectively to solve the problem      | Applies appropriate concepts with minor errors       | Limited application of concepts                | Incorrect or no application of concepts           |
| Solution Design                     | 25%       | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps        | Basic solution with limited feasibility        | Unclear or impractical solution                   |
| Use of Tools                        | 15%       | Effective use of modern tools, technologies, and real data                   | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools           | Minimal or incorrect use of tools                 |



|          |     |                                                                     |                                          |                                        |                     |
|----------|-----|---------------------------------------------------------------------|------------------------------------------|----------------------------------------|---------------------|
| Analysis | 10% | Thorough analysis with validated results and performance evaluation | Good analysis with reasonable validation | Basic analysis with limited validation | Poor or no analysis |
|----------|-----|---------------------------------------------------------------------|------------------------------------------|----------------------------------------|---------------------|